

Experiment - 5

Aim: Implementation of a knapsack problem using dynamic programming.

Knapsack:

Algorithm:

```
Input : number of items n, weight w[], value v[] & capacity [w]
Output : DP table, maximum value obtained, items that are selected.

step1 : start
step2 : Read the number of items n & capacity w
step3 : Read the weight & value of every item.
step4 : initialize:
    dp[i][0] = 0 for all items, & dp[0][j] = 0 for all capacity
step5 : for every item i=1 to n:
    → for every capacity j=1 to w:
        if w[i] <= j:
            → calculate value when the item is taken
              v[i] + dp[i-1][j-w[i]]
            → calculate value when the item is not taken
              dp[i-1][j]
            → store the maximum of these two values in dp[i][j].
        → otherwise
            → copy dp[i-1][j].
step6 : dp[n][w] gives maximum value
step7 : start from dp[n][w] and move backwards through the table to find selected items.
step8 : print results
step9 : stop
```

Program:

```
#include<iostream>

using namespace std;

class Knapsack {

private:

    int w[10];

    int v[10];

    int n;

    int W;

    int dp[10][100];

public:

    void getInput() {

        cout << "Enter number of items  : ";

        cin >> n;

        cout << "Enter knapsack capacity : ";

        cin >> W;

        for(int i = 1; i <= n; i++) {

            cout << "Item " << i << " weight and value : ";
```

```
cin >> w[i] >> v[i];

}

}

void solve() {

    for(int i = 0; i <= n; i++) dp[i][0] = 0;

    for(int j = 0; j <= W; j++) dp[0][j] = 0;

    for(int i = 1; i <= n; i++) {

        for(int j = 1; j <= W; j++) {

            if(w[i] <= j) {

                dp[i][j] = max(

                    v[i] + dp[i-1][j-w[i]],

                    dp[i-1][j]

                );

            } else {

                dp[i][j] = dp[i-1][j];

            }

        }

    }

}
```

```
}  
  
void printTable() {  
  
    cout << "\nDP Table:" << endl;  
  
    cout << "    ";  
  
    for(int j = 0; j <= W; j++)  
  
        cout << j << " ";  
  
    cout << endl;  
  
    for(int i = 0; i <= n; i++) {  
  
        cout << "i=" << i << " ";  
  
        for(int j = 0; j <= W; j++)  
  
            cout << dp[i][j] << " ";  
  
        cout << endl;  
  
    }  
  
}  
  
void findItems() {  
  
    cout << "\nSelected Items:" << endl;  
  
    int j = W;  
  
    for(int i = n; i >= 1; i--) {
```

```
if(dp[i][j] != dp[i-1][j]) {  
  
    cout << "Item " << i  
  
        << " (w=" << w[i]  
  
        << ", v=" << v[i]  
  
        << ") TAKEN " << endl;  
  
    j = j - w[i];  
  
} else {  
  
    cout << "Item " << i  
  
        << " (w=" << w[i]  
  
        << ", v=" << v[i]  
  
        << ") NOT taken " << endl;  
  
}  
  
}  
  
}  
  
void showResult() {  
  
    cout << "\nMaximum Value = "  
  
        << dp[n][W] << endl;  
  
}
```

```
};  
  
int main() {  
  
    Knapsack k;  
  
    k.getInput();  
  
    k.solve();  
  
    k.printTable();  
  
    k.showResult();  
  
    k.findItems();  
  
    return 0;  
  
}
```

Output:

```
"C:\Users\k bharath kumar\O... x + v
Enter number of items : 4
Enter knapsack capacity : 8
Item 1 weight and value : 2
3
Item 2 weight and value : 3
5
Item 3 weight and value : 4
6
Item 4 weight and value : 5
7

DP Table:
  0  1  2  3  4  5  6  7  8
i=0 0  0  0  0  0  0  0  0  0
i=1 0  0  3  3  3  3  3  3  3
i=2 0  0  3  5  5  8  8  8  8
i=3 0  0  3  5  6  8  9  11 11
i=4 0  0  3  5  6  8  9  11 12

Maximum Value = 12

Selected Items:
Item 4 (w=5, v=7) TAKEN
Item 3 (w=4, v=6) NOT taken
Item 2 (w=3, v=5) TAKEN
Item 1 (w=2, v=3) NOT taken

Process returned 0 (0x0)   execution time : 34.977 s
Press any key to continue.
|
```

Time complexity:

for (int i=1; i<N; i++) { — n
for (int j=1; j<=W; j++) { — n*W
if (W[i] <= j) { — n*W
dp[i][j] = max(
v[i] + dp[i-1][j-W[i]] - n*W
dp[i-1][j] — n*W
);
}
else {
dp[i][j] = dp[i-1][j]; — n*W
}
}
}

$\Rightarrow T(n) =$
 $n + n(W+1) + nW + \dots$
 $= n + nW + n + nW$
 $= 5nW + 2n$
 $= O(nW)$
 $\therefore T(n, W) = O(nW)$

Best case :- ex:- $n=3, W=5$ $\begin{cases} W[1]=10 \\ W[2]=20 \\ W[3]=30 \end{cases}$
 $\Rightarrow W[i] > j$
if condition = false $\Rightarrow T(n, W) = n + n(W+1) + nW$
 $= n + nW + n + nW$
 $= 5nW + 2n$
Best case = $O(nW)$

Average case :- ex:- $n=3, W=5$ $\begin{cases} W[1]=1 \\ W[2]=3 \\ W[3]=7 \end{cases}$
 $\Rightarrow$ ① $W[i] <= j$
② $W[i] > j$
if and else both execute $\Rightarrow T(n, W) = n + n(W+1) + nW$
 $= n + nW + n + nW$
 $= 5nW + 2n$
But loop runs completely Avg. case = $O(nW)$

Worst case :- ex:- $n=3, W=3$ $\begin{cases} W[1]=1 \\ W[2]=2 \\ W[3]=3 \end{cases}$
 $\Rightarrow W[i] <= j$
if condition = true $\Rightarrow T(n, W) = n + n(W+1) + nW + nW + nW$
 $= n + nW + n + 3nW$
 $= 5nW + 2n$
Worst case = $O(nW)$

Conclusion:

As per the experiment to Implementation of a knapsack problem using dynamic programming., Output was verified successfully according to the Experiment.